

1 Goals

Model can predict new edges in a graph over time.

- This will likely involve merging a GCN (likely attention-based) with a recurrent model, likely an LSTM.

2 Notes

Model will be created with a global node pool. Some nodes will be unconnected. This is ok.

3 Text Representation

First need to consider how to represent the text in edges and nodes.

3.1 Edge Embeddings

- Need to be a limited subset.
- Embeddings will be learned by the model.
- Example: born, died, friends, enemies, killed_by, etc.

3.2 Node Text Representation

3.2.1 Embedding Layer

- Generate separate embeddings for different characteristics using a text $\rightarrow$ vector model:
 - name
 - description
 - class
- Embeddings will be merged in an embedding layer in the model.

The best idea is to go with multiple pretrained embeddings of names, class, and description. Then put an embedding head that takes in each embedding vector, synthesizes it and projects down into GNN dimension.

- Dataset will be a concatenation of the three because we can just leave it as the concatenation or project down.

Text embedding could also be merged with structural embedding in some way, but the structure will probably be learned via convolutions so it is probably ok.

With embedding taken care of each node will be represented by an embedding and we have the starting structure for the graph.

3.2.2 Embeddings Layer

- Simple MLP that takes all the embedding vectors representing a model and projects them into hidden dimension.

4 Update Rule

$$h_i^{(t+1)} = \sigma \left(\sum_j a_{ij} \cdot m_{ij} \right)$$

where a_{ij} is an attention score and m_{ij} is the message.

4.1 Attention

$$a_{ij} = \text{softmax}(\sigma(a(z_i, z_j, \text{edge}_{ij})))$$

where:

$$z = W_n h$$

- Computed only over neighbors.

4.2 Messages

$$m_{ij} = W_h h_j^{(t)} + W_r \text{edge}_{ij}$$

4.3 Multi-Head Attention

- Multi-head attention should be considered after single head is finished.
- Basically the same thing but project down and concatenate.
- Dimension must be divisible by number of heads.
- Each head has separate W_h , W_r , etc.
- At the end concatenate outputs.
- For efficiency can share weights across heads.

5 Further Considerations

- Since we are using summation, should use layer normalization and residual connections.

$$h^{(t+1)} = \text{GNNLayer}(h^{(t)})$$

$$h^{(t+1)} = \text{LayerNorm}(h^{(t+1)} + h^{(t)})$$

$$h^{(t+1)} = \text{LayerNorm}(h^{(t+1)} + \text{MLP}(h^{(t+1)}))$$

- Dropout could also be useful for generalization.

6 Temporal Modeling

6.1 LSTM Hidden State

- Store a hidden state for each node.
- Use normal LSTM update conditioned on node hidden state.
- Stores fine-grained information.
- Use this for scoring.

7 Link Prediction

Predict only changes.

- All current triples are passed forward.
- Model predicts additions and deletions.

7.1 Scoring Heads

- Separate scoring heads for addition and deletion.

7.2 Candidate Generation

- Candidates are true positives plus equal number of sampled negatives.

7.2.1 Addition Negatives

- Node sampling
- Random
- Neighbors (depth 1–3)
- Embedding similarity (dot product)

7.2.2 Edge Sampling

- Random
- Frequency-based sampling

7.3 Candidate Scoring

$$\text{MLP}(i, j, \text{edge}, i_{\text{hidden}}, j_{\text{hidden}})$$

- Condition on LSTM hidden states
- Apply sigmoid

7.4 Feature Vector for Addition Head

7.4.1 Pair-level features

- $\text{pair_ever_seen} = 1[(u, v) \text{ seen}]$
- $\text{pair_recency} = 1/(1 + (t - \text{last_seen}))$
- $\text{pair_total_count_log} = \log(1 + x)$
- $\text{pair_recent_count_log} = \log(1 + x)$
- $\text{pair_last_streak_log} = \log(1 + x)$

7.4.2 Triple-level features

- $\text{triple_ever_seen} = 1[(u, r, v) \text{ seen}]$
- $\text{triple_recency} = 1/(1 + (t - \text{last_seen}))$
- $\text{triple_total_count_log} = \log(1 + x)$
- $\text{triple_recent_count_log} = \log(1 + x)$
- $\text{triple_last_streak_log} = \log(1 + x)$

7.4.3 Node-level features

- $\text{src_activity_total_log} = \log(1 + x)$
- $\text{dst_activity_total_log} = \log(1 + x)$
- $\text{src_activity_recent_log} = \log(1 + x)$
- $\text{dst_activity_recent_log} = \log(1 + x)$

7.4.4 Structural features

- $\text{common_neighbors_log} = \log(1 + |N_t(u) \cap N_t(v)|)$
- $\text{recent_two_hop_log} = \log(1 + |A \cap B|)$

8 Loss

Binary cross entropy:

$$\mathcal{L}_{BCE} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

- Positive edges are labeled 1
- Negative edges are labeled 0
- Losses are summed for addition and deletion

9 Generation

- Autoregressive generation
- Sample candidates based on probabilities
- Apply changes and feed back in